FleetTrack — Informe final de calidad y decisión GO / NO-GO

Curso 048 Aseguramiento de la Calidad del Software, Universidad Mariano Gálvez. Consolidado por Alejandro con los aportes de Manuel y Andrea. Fecha de las mediciones: 2026-11-05. Ambiente: QA local con Docker Compose y emulador Android 15 (API 35). Commit de referencia: rama `alejandro` sobre `main` (PR #16).

1. Resumen ejecutivo

FleetTrack funciona de punta a punta: dashboard web, API, mensajería con outbox, consumidor idempotente y app Android con modo sin conexión. Las **77 verificaciones automatizadas pasan**, la app Android se validó en un emulador real, el **pipeline de Jenkins se ejecutó de punta a punta** y las pruebas de carga a escala completa pasaron con holgura. El Quality Gate de SonarQube está en **OK** (0 bugs, 0 vulnerabilidades) con cobertura medida del backend de **82,1 %**.

Recomendación: NO-GO para el despliegue final por ahora; el candidato `v0.9.0-rc1` es apto en funcionalidad. El plan establece que un reporte obligatorio ausente deja la ejecución incompleta y sin aprobación final. Faltan dos cierres que dependen de personas (sección 8). Con ellas cerradas la decisión pasa a GO sin cambios en el producto.

2. Resultados de pruebas

Nivel	Herramienta	Resultado
Unitarias	xUnit	6 / 6
Integración y API	xUnit contra el stack real	22 / 22
Recuperación ante fallos	`tests/recovery/run-all.sh`	26 / 26
Funcionales, regresión y accesibilidad web	Cypress (+ axe-core)	15 / 15
Móvil	Maestro sobre emulador	8 / 8
Manual Android	Entrega fallida, mensaje sin conexión, evidencia	Aprobado
Total automatizado		**77 / 77**

Las suites de integración y recuperación también se ejecutaron con la API instrumentada para medir cobertura, con el mismo resultado (22/22 y 26/26).

3. Seguridad

Medida	Resultado
Semgrep (sustituto de Fortify): backend, frontend, mobile	0 hallazgos en los tres
SonarQube: bugs / vulnerabilidades / hotspots	0 / 0 / 0
OWASP ZAP web (baseline)	0 FAIL, 3 avisos informativos aceptados
OWASP ZAP API	0 FAIL, 0 avisos
npm audit y NuGet (`--vulnerable`)	0 vulnerabilidades
OWASP Dependency-Check (229 dependencias, base NVD con 420 399 registros)	Sin supresión: 39 hallazgos (3 críticos) **todos en un solo paquete**, `sqlite-net-pcl`. Es un **falso positivo por confusión de CPE** (ver abajo). Con la supresión documentada: 0 hallazgos
Contenedores	Todos sin privilegios

Análisis del hallazgo de Dependency-Check. `sqlite-net-pcl` 1.11.285 es un envoltorio ORM de C#, no el motor SQLite. La herramienta lo emparejó con `cpe:2.3:a:sqlite:sqlite:1.11.285` (confianza MEDIUM, solo por el nombre) y trató 1.11.285 como versión del motor, por lo que le atribuyó CVE de versiones antiguas (2015-2020). El motor que la app incluye es `SourceGear.sqlite3` 3.53.3 (por `SQLitePCLRaw.provider.e_sqlite3` 3.0.3), que la herramienta no marca. Se registró como **riesgo aceptado con justificación y caducidad 2027-01-31** en `tests/security/dependency-check-suppressions.xml`; el reporte original sin supresión se conserva en `reports/dependency-check/dependency-check-report-sin-supresion.*`. La aceptación la decide Alejandro.

Autorización: el conductor no puede operar rutas ni entregas ajenas (pruebas de integración) y los listados de administración devuelven 401 sin token y 403 para el rol conductor.

4. Calidad de código

Medida	Valor
Quality Gate	**OK**
Cobertura del backend (unitarias + integración + recuperación)	82,1 %
Bugs / vulnerabilidades / code smells	0 / 0 / 4 (aviso informativo de logging CA1873)
Duplicación	0 %

La cobertura de código se mide sobre el backend (C#, xUnit). Quedan fuera del cálculo, y la decisión consta en `tests/coverage/sonar-scan.sh`: las migraciones de EF (código generado), el frontend y la app móvil (su interfaz se valida con Cypress, axe y Maestro, no con cobertura de líneas) y los scripts, documentos e infraestructura. El umbral del Quality Gate no se modificó. La condición de cobertura de código nuevo no tiene líneas que medir en este análisis (el código nuevo es de pruebas y herramientas), por lo que no

bloquea; la cobertura global del backend (82,1 %) supera el 80 % del umbral aunque esa condición no se evalúe sobre código nuevo.

5. Rendimiento

k6 sobre `POST /api/tracking`, **a la escala del plan**, el 2026-11-06 en el equipo de QA (Intel Core i7-13620H, 10 núcleos, 15,6 GB de RAM; API, PostgreSQL, Redis, RabbitMQ y k6 en la misma máquina, emulador cerrado):

Escenario	Usuarios virtuales	Duración	Peticiones	p95	Máximo	Errores
Load	100	10 min	12 001	**20,0 ms**	675 ms	1 de 12 001 (0,008 %)
Stress	hasta 300	12 min	31 277	**23,1 ms**	160 ms	0
Spike	salto a 300	3,5 min	4 726	**29,7 ms**	80 ms	0

Criterio de salida (p95 < 500 ms y errores < 1 %): **cumple** en los tres. En el stress no se llegó a saturar el sistema con 300 usuarios virtuales: el límite real queda por encima de lo que el plan pide probar.

Sin pérdida ni duplicados: tras las tres corridas había 48 000 filas en `TrackingEvents` (48 000 identificadores distintos) y 48 000 eventos `LocationUpdated` procesados por el consumidor, igual a las 48 001 peticiones enviadas menos la única fallida; el outbox, la cola principal y la cola de fallidos quedaron en 0.

Limitaciones honestas: todos los usuarios virtuales reportan como el mismo conductor del seed (atajo declarado en `tests/k6/load.js`) y generador y servidor comparten máquina, así que la medición es válida para este equipo y no extrapolable a otro entorno.

6. Defectos

ID	Descripción	Severidad	Estado
D-01	Android 9+ bloqueaba HTTP: no había login móvil	Alta	Corregido (`network_security_config`)
D-02	La cámara de evidencia no abría (faltaba `<queries>`)	Alta	Corregido
D-03	Mensaje engañoso al finalizar sin red	Media	Corregido
D-04	Experimento E2 inestable tras otras suites	Media	Corregido
D-05	Sonar: 2 bugs, 5 vulnerabilidades, 14 smells	Media	Corregido
D-06	ZAP: 10 avisos de cabeceras	Media	Corregido (3 informativos aceptados)

ID	Descripción	Severidad	Estado
D-07	Flujos Maestro con teclado, cámara y patrones incorrectos	Baja	Corregido
D-08	El mensaje "guardada" no se limpia tras sincronizar	Baja	Corregido: el aviso se limpia al confirmarse en el servidor
D-09	Quality Gate en ERROR por cobertura no medida	Alta	Corregido: cobertura medida con `dotnet-coverage`
D-10	Dependency-Check sin clave de NVD	Media	Corregido: ejecutado con clave; hallazgo analizado como falso positivo
D-16	`sqlite-net-pcl`: 39 CVE atribuidos por confusión de CPE	Media	Riesgo aceptado con justificación (pendiente de aprobación de Alejandro)
D-11	Sonar marcaba `allowBackup` y HTTP en la app móvil	Media	Corregido: `allowBackup=false`, cleartext solo a `10.0.2.2`
D-12	Controlador de plantilla `WeatherForecast` sin uso	Baja	Corregido: eliminado
D-13	Semgrep local bloqueado por el Control de aplicaciones de Windows	Media	Corregido: corre en su imagen oficial de Docker
D-14	Experimento E4 usaba archivos temporales que fallaban bajo Jenkins en Windows	Media	Corregido: captura en variable
D-15	Script de cobertura no compilaba las pruebas de integración en un workspace limpio	Media	Corregido
D-18	La app seguía reportando ubicación después de finalizar la entrega, y el dashboard mostraba al conductor «En vivo»	Media	Corregido: solo reporta mientras hay una entrega en curso
D-17	Rediseño de la interfaz: axe halló contraste insuficiente en tema oscuro (4,09:1) y una animación que bajaba la opacidad del texto	Media	Corregido (5,86:1; animación solo de desplazamiento)

Ningún defecto crítico o alto sigue abierto.

7. Riesgos residuales

Riesgo	Tratamiento
Fortify no está disponible; se usó Semgrep	Pedir al docente una excepción académica expresa
La app no se probó en un teléfono físico	Requiere la IP de la PC en `AppConfig.cs` y en la `network_security_config`
Terraform y Ansible sin ejecutar contra Azure	Requiere confirmar Azure for Students
Jenkins corre solo en la máquina de desarrollo (controlador y agentes `qa` / `android` son el mismo equipo)	Aceptable para el curso; un equipo dedicado reduciría la interferencia de otras tareas

Riesgo	Tratamiento
El Control de aplicaciones de Windows bloquea de forma intermitente binarios recién compilados (afectó a Semgrep y al espacio de trabajo de Jenkins en `C:\Users`)	Se movió el espacio de trabajo a la unidad del proyecto y Semgrep corre en Docker; no se modificó ninguna configuración de seguridad
Sin OpenAPI publicado: ZAP API solo analiza `/health`	Publicar la especificación en QA

8. Decisión GO / NO-GO

Criterio de salida	Estado
Cero pruebas críticas fallidas	Cumple (77 / 77)
Quality Gate aprobado sin cambiar umbrales	Cumple
Sin vulnerabilidades críticas no aceptadas	**Cumple** si Alejandro acepta el falso positivo documentado de `sqlite-net-pcl`
Reportes obligatorios presentes	**Cumple** (Dependency-Check ejecutado el 2026-11-06)
p95 < 500 ms y errores < 1 % a escala completa	**Cumple** (p95 20 / 23 / 29,7 ms; 0,008 % de errores en load)
Usabilidad ≥ 80 % con 5 participantes	**No demostrado**: sin participantes
Pipeline de Jenkins ejecutado con el reporte de Quality Gate	**Cumple**: build #12 en verde de punta a punta (21 minutos; incluye Dependency-Check con clave de NVD, Maestro 8/8, ZAP, k6 y Quality Gate OK). El build #11, sin la clave, se detuvo en los criterios de salida, como debe. El despliegue final solo corre en `main` con la aprobación de Alejandro
Autorización explícita de Alejandro en Jenkins	Pendiente

Decisión recomendada: NO-GO provisional, a falta de dos cierres, ambos de personas:

1. Hacer la sesión de usabilidad con 5 personas y registrar `docs/usabilidad/plantilla-resultados.csv` (criterio: 80 % de éxito).
2. Alejandro aprueba o rechaza los riesgos residuales de la sección 7 (incluida la supresión de `sqlite-net-pcl`); después crea la etiqueta `v0.9.0-rc1` y autoriza el despliegue.

Con el 80 % de usabilidad y la aprobación, la decisión pasa a **GO** sin cambios en el producto.

9. Evidencia y reproducibilidad

- Matriz de riesgos: `docs/matriz-riesgos.xlsx` (14 riesgos con probabilidad, impacto y pruebas asociadas).

- Casos de prueba con trazabilidad a requisito, PR, prueba, resultado, evidencia y defecto: `docs/casos-prueba.xlsx` (hojas Casos, Requisitos y Defectos).
- Plan de pruebas y arquitectura: `docs/plan-pruebas.pdf`, `docs/arquitectura.pdf`.
- Jenkins: `infrastructure/jenkins/start-jenkins.sh` (controlador, plugins, credenciales y job como código) y `Jenkinsfile`; guía en `docs/guia-cierre-evidencias.md`.
- Cobertura: `bash tests/coverage/run-coverage.sh` y `bash tests/coverage/sonar-scan.sh`. Reportes en `reports/` (ignorada por Git; cada ejecución del pipeline los archiva por número de build y commit).
- Diseño de la interfaz web y móvil: `docs/diseno-interfaz.md`.
- Regeneración de documentos: `node docs/build-pdfs.mjs` y `python docs/src/build-xlsx.py`.